

Projet : Gestion d'un système de vente conditionnelle

Avril 2026

1 Diagramme entité-association

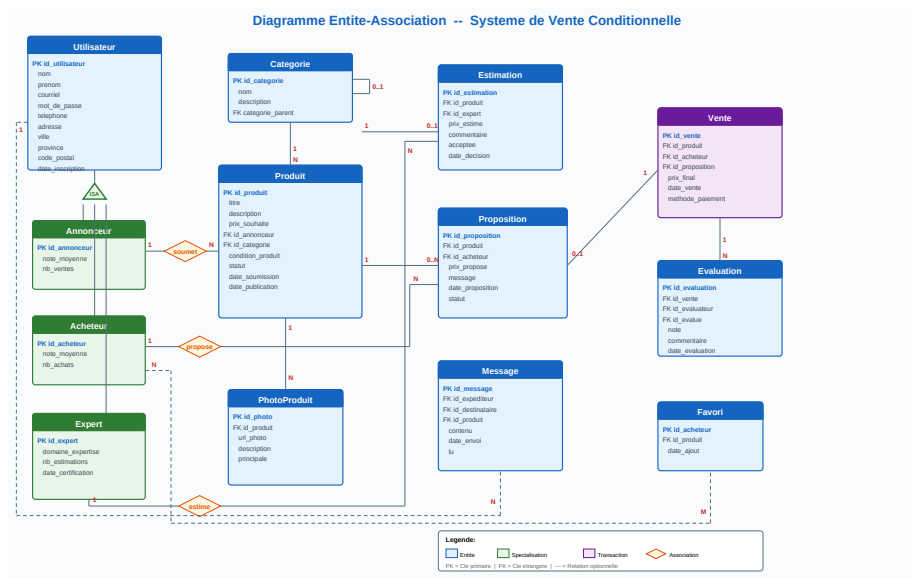


Figure 1: Diagramme entité-association du système de vente conditionnelle

2 Schéma relationnel

- **UTILISATEUR**(id_utilisateur, nom, prenom, courriel, mot_de_passe, telephone, adresse, ville, province, code_postal, date_inscription)
- **ANNONCEUR**(#id_annonceur, note_moyenne, nb_ventes)
- **ACHETEUR**(#id_acheteur, note_moyenne, nb_achats)

- **EXPERT**(#id_expert, domaine_expertise, nb_estimations, date_certification)
- **CATEGORIE**(id_categorie, nom, description, #categorie_parent)
- **PRODUIT**(id_produit, titre, description, prix_souhaite, #id_annonceur, #id_categorie, condition_produit, statut, date_soumission, date_publication)
- **ESTIMATION**(id_estimation, #id_produit, #id_expert, prix_estime, commentaire, acceptee, date_decision)
- **PROPOSITION**(id_proposition, #id_produit, #id_acheteur, prix_propose, message, date_proposition, statut)
- **VENTE**(id_vente, #id_produit, #id_acheteur, #id_proposition, prix_final, date_vente, methode_paiement)
- **EVALUATION**(id_evaluation, #id_vente, #id_evaluateur, #id_evalue, note, commentaire, date_evaluation)
- **PHOTOPRODUIT**(id_photo, #id_produit, url_photo, description, principale)
- **MESSAGE**(id_message, #id_expéditeur, #id_destinataire, #id_produit, contenu, date_envoi, lu)
- **FAVORI**(#id_acheteur, #id_produit, date_ajout)

3 Explications du code DDL

3.1 Clés primaires

Chaque table possède une clé primaire qui permet d’identifier de façon unique chaque ligne. Pour la plupart des tables, on a utilisé un identifiant entier comme `id_utilisateur`, `id_produit`, `id_vente`, etc.

Pour les tables `annonceur`, `acheteur` et `expert`, on utilise le même identifiant que dans `utilisateur`. Cela correspond à la spécialisation du modèle EA.

Dans la table `favori`, la clé primaire est composée de `id_acheteur` et `id_produit` afin d’éviter les doublons.

3.2 Clés étrangères

Les clés étrangères servent à relier les tables entre elles. Par exemple, un produit est lié à un annonceur, donc `id_annonceur` dans la table `produit` référence la table `annonceur`. De la même façon, une proposition est liée à un produit et à un acheteur.

Ces contraintes permettent d’éviter d’avoir des valeurs qui ne correspondent à rien dans les autres tables. Par exemple, on ne peut pas insérer une proposition avec un produit qui n’existe pas.

3.3 Contraintes d’unicité

On a utilisé `UNIQUE` dans certains cas où une valeur ne devrait pas être répétée. Par exemple, le courriel d’un utilisateur doit être unique. De la même façon, le nom d’une catégorie est unique.

3.4 Types de données

On utilise des types simples comme `INTEGER`, `VARCHAR`, `NUMERIC`, `DATE` et `TIMESTAMP`.

3.5 Organisation des tables

Les tables ont été séparées pour éviter la redondance. Par exemple, `utilisateur` contient les informations générales et les rôles sont séparés.

3.6 Valeurs par défaut

Certaines colonnes ont des valeurs par défaut. Par exemple, les dates utilisent la date actuelle, et certains champs comme les compteurs commencent à 0. Ça simplifie les insertions.

4 Requêtes SQL et résultat attendu

4.1 Requête 1

```
SELECT p.titre, p.prix_souhaite, c.nom, u.prenom, u.nom
FROM produit p
JOIN categorie c ON p.id_categorie = c.id_categorie
JOIN annonceur a ON p.id_annonceur = a.id_annonceur
JOIN utilisateur u ON a.id_annonceur = u.id_utilisateur
WHERE p.statut = 'publie';
```

Affiche les produits publiés avec leur catégorie et le nom de l'annonceur.

4.2 Requête 2

```
SELECT c.nom, COUNT(*)
FROM produit p
JOIN categorie c ON p.id_categorie = c.id_categorie
GROUP BY c.nom;
```

Affiche le nombre de produits par catégorie.

4.3 Requête 3

```
SELECT u.prenom, u.nom, COUNT(*)
FROM proposition pr
JOIN acheteur a ON pr.id_acheteur = a.id_acheteur
JOIN utilisateur u ON a.id_acheteur = u.id_utilisateur
GROUP BY u.prenom, u.nom;
```

Classe les acheteurs selon le nombre de propositions.

4.4 Requête 4

```
SELECT p.titre, p.prix_souhaite, v.prix_final
FROM vente v
JOIN produit p ON v.id_produit = p.id_produit;
```

Compare le prix souhaité et le prix final.

4.5 Requête 5

```
SELECT p.titre, c.nom, e.prix_estime, u.prenom, u.nom
FROM produit p
JOIN categorie c ON p.id_categorie = c.id_categorie
JOIN estimation e ON p.id_produit = e.id_produit
JOIN expert ex ON e.id_expert = ex.id_expert
JOIN utilisateur u ON ex.id_expert = u.id_utilisateur;
```

Affiche les estimations et les experts.

4.6 Requête 6

```
SELECT pr.prix_propose, p.titre, ua.prenom, uv.prenom
FROM proposition pr
JOIN produit p ON pr.id_produit = p.id_produit
JOIN acheteur a ON pr.id_acheteur = a.id_acheteur
JOIN utilisateur ua ON a.id_acheteur = ua.id_utilisateur
JOIN annonceur an ON p.id_annonceur = an.id_annonceur
JOIN utilisateur uv ON an.id_annonceur = uv.id_utilisateur;
```

Affiche les propositions avec acheteur et annonceur.

4.7 Requête 7

```
SELECT v.id_vente, p.titre, v.prix_final
FROM vente v
JOIN produit p ON v.id_produit = p.id_produit;
```

Affiche les ventes.

4.8 Requête 8

```
SELECT ev.note, p.titre
FROM evaluation ev
JOIN vente v ON ev.id_vente = v.id_vente
JOIN produit p ON v.id_produit = p.id_produit;
```

Affiche les évaluations.

4.9 Requête 9

```
SELECT p.titre, u.prenom
FROM favori f
JOIN produit p ON f.id_produit = p.id_produit
JOIN acheteur a ON f.id_acheteur = a.id_acheteur
JOIN utilisateur u ON a.id_acheteur = u.id_utilisateur;
```

Affiche les favoris.

4.10 Requête 10

```
SELECT p.titre, COUNT(*)
FROM message m
JOIN produit p ON m.id_produit = p.id_produit
GROUP BY p.titre;
```

Compte les messages par produit.